JIT2311- OBJECT ORIENTED PROGRAMMING LABORATORY

Lab exercises:

1(a)Develop a C++ code to create a class and object and print the member variables

PROGRAM:

```
#include<iostream>
#include<string>
using namespace std;
class Student
{
private:
int age;
String Name;
int Rollno;
public:
Student(String n,int a, int r)
{
Name=n;
age=a;
RollNo=r;
}
void displayDetails()
{
cout<<"NAME:"<<Name<<endl;
```

```
cout<<”AGE:”<<age<<endl;
cout<<”ROLLNO:”<<RollNo<<endl;
}
};
int main(){
Student s(“JOHN”,18,20);
s.displayDetails();
cout<<endl;
return 0;
}
```

---

b) Develop a C++ program to show the working of default constructor, parametrized constructor and copy constructor and destruct any object.

PROGRAM:

```
#include <iostream>
#include <string>
using namespace std;
class Student {
private:
   int rollno;
   float percentage;
public:
   // Default constructor
   Student() {
```

```
        rollno = 0;
        percentage = 0.0f;
    }

    // Parameterized constructor
    Student(int r, float p) {
        rollno = r;
        percentage = p;
    }

    // Copy constructor
    Student(Student &c) {
        rollno = c.rollno;
        percentage = c.percentage;
    }

    // Method to display student details
    void display() {
        std::cout << "Roll No: " << rollno << std::endl;
        std::cout << "Percentage: " << percentage << std::endl;
    }
};
```

```
int main() {
 // Using default constructor
 Student s2;
 std::cout << "Student  (default constructor):" << std::endl;
 s2.display();
std::cout << std::endl;
// Using parameterized constructor
Student s3(101, 85.5);
std::cout << "Student (parameterized constructor):" << std::endl;
 s3.display();
 std::cout << std::endl;
// Using copy constructor
Student s4= s3; // or Student s3(s2);
 std::cout << "Student 3 (copy constructor):" << std::endl;
 s4.display();
 std::cout << std::endl;
 ~student()
{
std::cout<<"Destructor called for:"<<rollno<<std::endl;
}
   return 0;
```

---

2. Implement a C++ code to demonstrate the concept of static member function.

PROGRAM:

```
#include<iostream>
using namespace std;
class Student
{
private:
Static int count;
int id;
int stdid;
int regno;
public:
Student(int i,int r)
{
stdid=i;
regno=r;
}
Student(){
count++;
id=count;
}
~Student()
{
count--;
```

```
}
Static int getCount()
{
return count;
}
void display()
{
std::cout<<”ENTER STUDENT DETAILS:”<<endl;
std::cout<<”STUDENT ID:”<<stdid<<endl<<”REG NUM:”<<regno<<endl;
std::cout<<”OBJECT ID:”<<id<<std::endl;
}
};
int student::count=0;
int main(){
Student obj1;
Student obj2;
Student obj3;
std::cout<<”NUMBER OF OBJECTS
CREATED:”<<Student::getCount<<std::endl;
obj1.display();
obj2.display();
obj3.display();
return 0;
}
```

3. Develop a menu driven C++ program to find area of two-dimensional objects (any three) using function overloading.

PROGRAM:

```
#include <iostream>
using namespace std;
class area
{
   public:
   int side;
   int length;
   int breadth;
   float pi;
   int radius;
   void squareprint()
   {
      cout<<"ENTER THE SIDES OF SQUARE:"<<side<<endl;

   }
   void rectangleprint()
   {
      cout<<"ENTER THE LENGTH AND BREADTH:"<<length<<breadth<<endl;

   }
   void circleprint()
```

```
    {
        cout<<"ENTER THE RADIUS:"<<radius<<endl;
        cout<<"ENTER THE PI VALUE:"<<pi<<endl;

    }
};
int main()
{
    int choice;
    float pi=3.14,r,area;
    area=pi*radius*2;
    int squareprint;
    {
        return(side*side);
    }
    int rectangleprint;
    {
    return(length*breadth);
    }
    int circleprint;
    {
        return(area);
    }
```

```
area a;
cout<<"1.SQUARE"<<endl;
cout<<"2.RECTANGLE"<<endl;
cout<<"3.CIRCLE"<<endl;
cout<<"ENTER YOUR CHOICE(1-3):"<<endl;
cin>>choice;
switch(choice)
{
   case 1:
   cout<<"AREA OF SQUARE :"<<squareprint()<<squareprint<<endl;
   break;
   case 2:
   cout<<"AREA OF RECTANGLE :"<<rectangleprint()<<rectangleprint<<endl;
   break;
   case 3:
   cout<<"AREA OF CIRCLE:"<<circleprint()<<circleprint<<endl;
   break;
   default:
   cout<<"INVALID CHOICE"<<endl;
   return 0;
}
}
```

4(a) Develop a C++ code to overload the unary operators using operator function

PROGRAM:

```
#include<iostream>
using namespace std;
class Overload
{
private:
int a;
int b;
int c;
int d;
public:
Overload(int f,int i)
{a = f;
  b = i;
}
void display()
{
cout <<"A:"<<a<<"B:"<<b<<endl;
}
Overload operator +()
{
a = +a;
```

```
b = +b;
c = a + b;
cout << "\nC: " <<c;
return overload (a,b);
}
Overload operator-()
{
a = - a;
b = - b;
d = a - b;
cout<<"\nD:"<d;
return Overload(a,b);
}
};
int main()
{
overload M1(5, 4) , M2(- 2, - 8) ;
-M1;
M1.display();
+M2;
M2.display();
return 0;
}
```

(b) Develop a C++ code to overload binary operators using operator function

PROGRAM:

```
#include <iostream>
#include <string.h>
using namespace std;
class AddString
{
public:
char s1[25], s2 [25];
AddString (char str1 [25], char str2 [25])
{
strcpy (this→s1,str1);
strcpy(this→s2,str2);
}
void operator +()
cout << "In CONCATENATION:" << strcat (s1,s2);
}
};
int main()
{
char str1[]="GOOD";
char str2[]" MORNING";
AddString a1 (str1, str2);
```

```
+a1;

return 0;

}
```

---

5. (a) **Implement *a C++ code to swap any two values using function template.***

PROGRAM:

INT:

```
#include<iostream>

using namespace std;

template <class T>

int  swap_numbers (T&x, T&y)

{

T t;

t=x;

x=y;

y=t;

return 0;

}

int a, b;

a=10 ,b=20;

swap_numbers(a,b);

cout << a <<"  "<<b<<endl;

return 0;

}
```

DOUBLE:

```
#include<iostream>
using namespace std;
template <class T>
int  swap_numbers (T&x, T&y)
{
T t;
t=x;
x=y;
y=t;
return 0;
}
double a, b;
a=10.5 ,b=20.5;
swap_numbers(a,b);
cout << a <<"  "<<b<<endl;
return 0;
}
```

STRING:

```
#include<iostream>
using namespace std;
template <class T>
string swap_numbers (T&x, T&y)
```

```
{
T t;
t=x;
x=y;
y=t;
return 0;
}
string a, b;
a=”APPLE”,b=”BALL”;
swap_numbers(a,b);
cout << a <<"  "<<b<<endl;
return 0;
}
```

CHAR:

```
#include<iostream>
using namespace std;
template <class T>
void swap (char &a, char &b)
{
char temp;
temp=a;
a=b;
b=temp;
```

```
}
int main()
{
char ch1, ch2;
cout << "Enter first character= ";
cin>> ch1;
cout <<"Enter second character= ";
cin>>ch2;
swap(ch1,ch2);
cout << "After swapping character: ";
cout << "\n Now first character: "<<ch1;
 cout <<< "\n Now second character: "<<ch2;
return 0;
}
```

(b) Implement a C++ code to find the maximum of any three values using a class template

PROGRAM:

```
#include <iostream>
using namespace std;
template <typename T>
T myMax (TX, Ty, Tz)
{
return (x>y)? x:y;
```

```
return (y>z)? y:z;
return (z>x)?z:x;
}
int main()
{
cout <<< myMax <int>(3,7, 2)<<endl;
cout << myMax<float>(3.0, 7.0,4.0)<<endl;
cout << myMax<char> ('R', 'V','A')<<endl;
cout << myMax<String>("R", "A", "V"<<endl;
return 0;
}
```

---

6. Develop a C++ code to handle division-by-zero and out-of-range exception.

PROGRAM TO HANDLE DIVISION BY ZERO:

```
#include <iostream>
#include <stdexcept>
using namespace std;
float Division (float num, float den)
{
if (den==0)
{
throw runtime error ("MATH ERROR: ATTEMPTED TO DIVIDE BY ZERO");
}
```

```
return (num/den);
}
int main()
{
float numerator, denominator, result;
numerator=12.5;
denominator = 0;
try
{
result = Division (numerator, denominator);
cout << "THE QUOTIENT IS: " << result << endl;
}
catch (runtime error &e)
{
cout << "EXCEPTION OCCURED" << endl <<e.what();
}
}
```

PROGRAM FOR INDEX-OUT-OF-RANGE:

```
#include <iostream>
#include <stdexcept>
using namespace std;
int main()
const int Maxsize=5;
```

```
int array [Maxsize] = {1,2,3,4,5};

int index = 0;

try

{

while (true)

{

if (index < MaxSize)

{

std::cout << "Array value at Index: " << index << "is" << array [index] <<
std::endl;

index++;

}

else

{

throw std:: Out of range ("Array index out of Exception!");

}

}

}

catch (const std::out_of_range &e)

{

 std:: cerr<<e.what () << std::endl;

}

return 0;

}
```

7. a) Implement a Java code to print the sum, multiply, subtract, divide and remainder of two numbers. Get the input from the user.

PROGRAM:

```
package calculator;

import java.util.Scanner;

public class Calculator {

public static void main(String[] args)

{

Scanner scanner = new Scanner(System.in);

System.out.println("Enter the first number: ");

double num1 = scanner.nextDouble();

System.out.println("Enter the second number: ");

double num2 = scanner.nextDouble();

double sum = num1 + num2;

double difference = num1 - num2;

double product = num1 * num2;

double quotient = num1 / num2;

double remainder = num1 % num2;

System.out.println("Sum: " + sum);

System.out.println("Difference: " + difference);

System.out.println("Product: " + product);

System.out.println("Quotient: " + quotient);

System.out.println("Remainder: " +

remainder);
```

```
// Close the scanner
scanner.close();
}
}
```

(b) Develop a Java program to print and return the current class instance using this keyword.

PROGRAM:

```
public class Rectangle {
// Instance variables
private double length;
private double width;
// Constructor
public Rectangle(double length, double width) {
this.length = length;
this.width = width;
}
// Method to print the current instance of the class and return it
public Rectangle getCurrentInstance() {
System.out.println("Current instance: " + this);
return this; // Returning the current instance
}
// Override the toString() method to provide a meaningful string
representation of the object
```

```
@Override
public String toString() {
return "Rectangle [length = " + length + ", width = " + width + "]";
}
// Method to display the rectangle's dimensions
public void displayDimensions() {
System.out.println("Length: " + this.length + ", Width: " + this.width);
}
}
```

---

8. Develop an application with Employee class with Emp name, Emp id. Address. Mail_id.Mobile no as members. Inherit the classes. Programmer. Assistant Professor. Associate Professor and Professor from employee class. Add Basic Pay (BP) as the member of all the inherited classes with 97% of BP as DA, 10% of BP as HRA, 12% of BP as PF. 0.1% of BP for staff club fund. Generate pay slips for the employees with their gross and net salary.

PROGRAM:

```
import java.util.Scanner;
class Employee {
String name;
int id;
int grossSalary, HRA, medical, transport, PF, tax;
public Employee(String name, int id, int grossSalary, int HRA, int medical, int transport, int PF, int
tax) {
```

```
this.name = name;
this.id = id;
this.grossSalary = grossSalary;
this.HRA = HRA;
this.medical = medical;
this.transport = transport;
this.PF = PF;
this.tax = tax;
}
void getDetails() {
System.out.println("Name: " + name);
System.out.println("ID: " + id);
System.out.println("Gross Salary: " + grossSalary);
System.out.println("HRA: " + HRA);
System.out.println("Medical Insurance: " + medical);
System.out.println("Transport: " + transport);
System.out.println("PF: " + PF);
System.out.println("Income Tax: " + tax);
}
String display() {
int benefits = calculateBenefits();
int basicSalary = grossSalary + benefits - (HRA + medical + transport + PF +
tax);
System.out.println("Basic Salary: " + basicSalary);
```

```
return "Basic Salary: " + basicSalary;
}
int calculateBenefits() {
return 0; // Default implementation, can be overridden
}
}
class Programmer extends Employee {
int hike;
public Programmer(String name, int id, int grossSalary, int HRA, int medical,
int transport, int PF, int
tax, int hike) {
super(name, id, grossSalary, HRA, medical, transport, PF, tax);
this.hike = hike;
}
@Override
int calculateBenefits() {
return hike;
}
void call() {
System.out.println("Programmer Salary");
getDetails();
display();
}
}
```

```
class AssistantProfessor extends Employee {

int researchAllowance;

public AssistantProfessor(String name, int id, int grossSalary, int HRA, int
medical, int transport, int

PF, int tax, int researchAllowance) {

super(name, id, grossSalary, HRA, medical, transport, PF, tax);

this.researchAllowance = researchAllowance;

}

@Override

int calculateBenefits() {

return researchAllowance;

}

void call() {

System.out.println("Assistant Professor Salary");

getDetails();

display();

}

}

class Professor extends Employee {

int publicationBonus;

public Professor(String name, int id, int grossSalary, int HRA, int medical, int
transport, int PF, int

tax, int publicationBonus) {

super(name, id, grossSalary, HRA, medical, transport, PF, tax);
```

```
this.publicationBonus = publicationBonus;
}
@Override
int calculateBenefits() {
return publicationBonus;
}
void call() {
System.out.println("Professor Salary");
getDetails();
display();
}
}
public class SalaryProgram {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Enter name: ");
String name = sc.nextLine();
System.out.print("Enter ID: ");
int id = sc.nextInt();
sc.nextLine(); // Consume newline left-over
System.out.print("Enter profession
(Programmer/AssistantProfessor/Professor): ");
String profession = sc.nextLine();
if (profession.equalsIgnoreCase("Programmer")) {
```

```
System.out.print("Enter Gross Salary: ");

int grossSalary = sc.nextInt();

System.out.print("Enter HRA: ");

int HRA = sc.nextInt();

System.out.print("Enter Medical Insurance: ");

int medical = sc.nextInt();

System.out.print("Enter Transport: ");

int transport = sc.nextInt();

System.out.print("Enter PF: ");

int PF = sc.nextInt();

System.out.print("Enter Tax: ");

int tax = sc.nextInt();

System.out.print("Enter Hike: ");

int hike = sc.nextInt();

Programmer p = new Programmer(name, id, grossSalary, HRA, medical,
transport, PF, tax,

hike);

p.call();

} else if (profession.equalsIgnoreCase("AssistantProfessor")) {

System.out.print("Enter Gross Salary: ");

int grossSalary = sc.nextInt();

System.out.print("Enter HRA: ");

int HRA = sc.nextInt();

System.out.print("Enter Medical Insurance: ");
```

```
int medical = sc.nextInt();

System.out.print("Enter Transport: ");

int transport = sc.nextInt();

System.out.print("Enter PF: ");

int PF = sc.nextInt();

System.out.print("Enter Tax: ");

int tax = sc.nextInt();

System.out.print("Enter Research Allowance: ");

int researchAllowance = sc.nextInt();

AssistantProfessor ap = new AssistantProfessor(name, id, grossSalary, HRA,
medical, transport,

PF, tax, researchAllowance);

ap.call();

} else if (profession.equalsIgnoreCase("Professor")) {

System.out.print("Enter Gross Salary: ");

int grossSalary = sc.nextInt();

System.out.print("Enter HRA: ");

int HRA = sc.nextInt();

System.out.print("Enter Medical Insurance: ");

int medical = sc.nextInt();

System.out.print("Enter Transport: ");

int transport = sc.nextInt();

System.out.print("Enter PF: ");

int PF = sc.nextInt();
```

```
System.out.print("Enter Tax: ");

int tax = sc.nextInt();

System.out.print("Enter Publication Bonus: ");

int publicationBonus = sc.nextInt();

Professor pr = new Professor(name, id, grossSalary, HRA, medical, transport,
PF, tax,

publicationBonus);

pr.call();

} else {

System.out.println("Invalid profession entered.");

}

sc.close();

}

}
```

---

9. Develop a Java program to create a 'Geometry package with relevant classes (For e-g. Circle/Square/Polygon and so on.) and export it.

PROGRAM:

```
//Rectangle

package Geometry

public class Rectangle {

int width=10;

int length=20;

public int area(){
```

```
return width*length;
}
}
//Square
package Geometry;
public class Square {
   int side=5;
   public int area(){
   return side*side;
}
}
//mainclass
package Mainpack;
import Geometry.*;
public class Mainclass {
public static void main(String[] args){
Rectangle rec=new Rectangle();
System.out.println("Rectangle Area:"+rec.area()
Square sq=new Square();
System.out.println("SQUARE AREA:"+sq.area());
}   }
```

---

10. Create an interface for 'Playable with classes Football. Volleyball and Basketball.

PROGRAM:

```
public interface Playable {
 void play();
 void country();
}
public class Basketball implements Playable
{
@Override
public void play(){
   System.out.println("THERE ARE 10 PLAYERS IN BASKETBALL!");
}
@Override
public void country(){
   System.out.println("THE LEADING COUNTRY IN BASKETBALL IS USA");
}
}
public class Football implements Playable
{
@Override
public void play(){
   System.out.println("THERE ARE 11 PLAYERS IN FOOTBALL!");
}
@Override
```

```
public void country(){
   System.out.println("THE LEADING COUNTRY IN FOOTBALL IS ARGENTINA");
}
}
public class Volleyball implements Playable {
   @Override
   public void play(){
      System.out.println("THERE ARE SIX PLAYERS IN VOLLEYBALL!");
   }
   @Override
   public void country(){
      System.out.println("THE LEADING COUNTRY IN VOLLEYBALL IS
POLAND");
   }
}
public class Main {
public static void main(String[] args){
   Playable bb=new Basketball();
   bb.play();
   Playable fb=new Football();
   fb.play();
   Playable vb=new Volleyball();
   vb.play();
bb.country();
```

```
fb.country();

vb.country();

}

}
```